

Clock2Q+: A Simple and Efficient Replacement Algorithm for Metadata Cache in VMware vSAN

Yiyan Zhai
Carnegie Mellon University
Pittsburgh, United States
yiyanz@andrew.cmu.edu

Bintang Dwi Marthen
Bandung Institute of Technology
Bandung, Indonesia
marthen.bintangdwi@gmail.com

Sarath Balivada
Broadcom Inc.
Bangalore, United States
sarath.balivada@broadcom.com

Vamsi Sudhakar Bojji
Broadcom Inc.
Bangalore, United States
vamsi-sudhakar.bojji@broadcom.com

Eric Knauff
Broadcom Inc.
Palo Alto, United States
eric.knauff@broadcom.com

Jitender Rohilla
Broadcom Inc.
Palo Alto, United States
jitender.rohilla@broadcom.com

Jiaqi Zuo
Broadcom Inc.
Palo Alto, United States
alvin.zuo@broadcom.com

Quanxing Liu
Broadcom Inc.
Palo Alto, United States
quanxing.liu@broadcom.com

Maxime Austruy
Broadcom Inc.
Pully, Switzerland
maxime.austruy@broadcom.com

Wenguang Wang
Broadcom Inc.
Palo Alto, United States
wenguang.wang@broadcom.com

Juncheng Yang
Harvard University
Cambridge, United States
juncheng@g.harvard.edu

ABSTRACT

Cache replacement algorithms are critical building blocks of storage systems. This paper examines the characteristics of metadata caches and argues that they inherently exhibit correlated references, even when the corresponding data accesses do not contain correlated references. The presence of correlated references reduces the effectiveness of cache replacement algorithms because these references are often mistakenly categorized as hot blocks.

Clock2Q+ is specifically designed for metadata caches and has been implemented in vSAN and VDFS, two flagship storage products of Broadcom Inc..

Similar to S3-FIFO, Clock2Q+ uses three queues; however, Clock2Q+ introduces a correlation window in the Small FIFO queue, where blocks in this window do not set the reference bit. This simple enhancement allows Clock2Q+ to outperform state-of-the-art replacement algorithms. Compared to S3-FIFO, the second-best performing algorithm, Clock2Q+ achieves up to a 28.5% lower miss ratio on metadata traces. Clock2Q+ possesses the essential properties required for large-scale storage systems: it has low CPU overhead on cache hits, low memory overhead, scales efficiently to multiple CPUs, and is both easy to tune and implement. Additionally, Clock2Q+ outperforms state-of-the-art cache replacement algorithms on data traces as well.

1 INTRODUCTION

Storage systems typically consist of multiple tiers, such as CPU caches, memory, and persistent media (e.g., SSDs and HDDs). Caches are widely employed to accelerate data access.

This paper examines block caches (also referred to as page caches, buffer caches, or buffer pools), which are common memory caches that manage fixed-size blocks (e.g., 4KB) to enhance storage access. In a block cache, the fast tier is memory, whereas the slow tier consists of various storage devices, such as SSDs and magnetic drives.

A key challenge in block caching is designing an efficient cache replacement algorithm that determines which data remain in the cache and which data are evicted. An ideal algorithm should aim to achieve:

- *High hit ratio* across diverse workloads, avoiding pathological cases that degrade the hit ratio.
- *Low CPU overhead* on cache hits, ensuring that cache access does not become a bottleneck, especially at high hit ratios (close to 100%).
- *Scalability to multiple CPUs*. The algorithm should scale efficiently on multi-core systems.
- *Minimal tuning required* for configuration parameters, enabling good performance across various workloads.
- *Ease of implementation and debugging*, reducing development effort and ensuring reliability.

Block caches store various types of data, which influence the design of the cache replacement algorithm:

- *Data Cache*: Stores the actual storage payload (e.g., file data).
- *Metadata Cache*: Stores metadata used to manage the storage system (e.g., B tree blocks).

Many systems maintain separate caches for data and metadata. For example, Linux uses the Buffer Cache to cache file system metadata and the Block Cache to cache file data [6]. vSAN, a distributed block storage system, includes both a

metadata cache and a data cache. vSAN features a small data cache at the DOM Client layer and multiple large metadata caches at its zDOM and LSOM2 layers [44, 45].

By analyzing the fundamental properties of data caches and the typical usage of metadata to manage data, this paper highlights key differences between them. In a metadata cache, some blocks may be referenced multiple times within a short period, followed by a long period of inactivity. Such frequent accesses within a short period, referred to as “correlated references”, should be treated as a single reference by the cache replacement algorithm. If these blocks are misclassified as hot due to the high number of references they receive in this window, they will remain in the cache for an extended period. A block exhibiting this reference pattern is still considered a cold block and should be evicted once its correlated reference period ends. This behavior differs from most data cache access patterns, where cold blocks are typically accessed only once. If this property is not accounted for, a cold block in the metadata cache could be mistakenly treated as a hot block, thereby reducing the overall hit ratio.

Most existing algorithms struggle to handle this access pattern effectively. For example, S3-FIFO [53] uses a Small FIFO queue to filter out unpopular blocks: if a block does not receive a request while in the Small FIFO queue, it is evicted early. However, this design is vulnerable to correlated accesses because such blocks are misclassified as hot and moved to the Main Clock, even though they will not receive any further requests after the correlation window ends. Therefore, tailoring the cache algorithm specifically for metadata workloads can provide additional performance benefits for the metadata cache.

In this paper, we describe a new cache replacement algorithm, Clock2Q+, that is designed for metadata cache and has been implemented in vSAN. Clock2Q+ is a hybrid of Clock2Q (§3.2) and S3-FIFO [53] by combining key design ideas of both algorithms and improving upon certain aspects of each. Clock2Q+ splits the small FIFO queue in S3-FIFO into two “virtual” segments, where requests to blocks in the first segment do not set the reference bit. Compared to Clock2Q, it reduces cache misses when hot blocks first enter the cache. Compared to S3-FIFO, it reduces cache misses caused by correlated references. This simple change allows Clock2Q+ to achieve up to a 28.5% lower miss ratio than S3-FIFO on metadata cache workloads. Moreover, Clock2Q+ also achieves lower miss ratio on data cache workloads even though they may not exhibit strong correlated access patterns. It also provides a low cache hit overhead, high scalability, and remarkable simplicity, which enables quick implementation in production and add other features such as concurrency and resizing. When handling dirty blocks, simple adjustments are made to the algorithm without compromising the cache miss ratio.

This paper has the following contributions:

- Identifies and establishes that correlated references are a common access pattern in metadata caches.
- Proposes a simple method to convert data traces into metadata traces. This simplifies the evaluation

of cache replacement algorithms on metadata cache workloads as most existing traces are collected from data accesses.

- Describes Clock2Q+, a scalable, low-overhead cache replacement algorithm implemented in vSAN.
- Evaluates Clock2Q+ on both metadata and data cache workloads, demonstrating that it outperforms state-of-the-art methods with lower miss ratios.

2 METADATA CACHE AND CORRELATED REFERENCES

2.1 Data Cache and Metadata Cache

A data cache is the most common type of cache where the cached data is the actual blocks (from a storage system). Most, if not all, existing caches and replacement algorithms are designed for storing data blocks.

In addition to data caches, metadata caches (also called index caches in many systems) are also widely used. Many storage systems use a form of ordered key-value store, such as B tree, to map the Logical Block Number (LBN) to the Physical Block Number (PBN). For example, VDFS, zDOM in vSAN ESA, LSOM2 in vSAN OSA, Linux XFS, and Btrfs all use B tree [32, 33, 46]; LSOM2 in vSAN ESA uses SplinterDB [12]; Unix FFS and Linux ext2/ext3 use direct and indirect block maps similar to page table; Linux ext4 uses extent tree [30]. In all these examples, the key-value store has fixed-size leaf blocks that store the LBN to PBN mapping. Because the key-value store is often large, only a small portion of it is cached in the metadata cache discussed in this work.

2.2 Correlated References

Storage often consists of multiple layers. Consider a two-layered system where an upper file system, such as ext4, is formatted on top of a disk array box that contains a lower file system. Within this lower file system, there are both a data cache and a metadata cache. The data cache stores common data requested by the upper file system, while the metadata cache stores the disk array’s own B tree blocks, which map the logical addresses of data received from the upper file system to physical addresses on disks. The data cache has a lower temporal locality because the upper file system also has a data cache that filters out repeated accesses [16, 48, 49]. However, as the following analysis shows, the metadata cache can still experience frequent, short-term accesses to the same metadata blocks, referred to as “correlated references”.

Using B tree as an example, assume the B tree uses LBN as the key and PBN as the value. Figure 1 shows some metadata blocks with address m_1, m_2 , etc., and data blocks with LBN L_1, L_2 , etc.

If the data trace accesses 4 LBNs in the following order: 1, 5, 107, and 720, the metadata accesses will be: $m_1, m_2, m_4, m_1, m_2, m_4, m_1, m_2, m_5, m_1, m_3, m_6$. It is evident that the non-leaf blocks (m_1, m_2, m_3) are accessed frequently. Any reasonable replacement algorithm will keep non-leaf blocks in the cache. Therefore, when analyzing metadata caches,

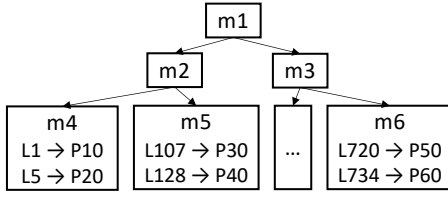


Figure 1: Example B tree for mapping LBN to PBN. Lookup LBN 5 will read metadata block $m1$, $m2$, $m4$ and find LBN 5 is mapped to PBN 20.

we ignore non-leaf blocks and focus solely on the leaf blocks, which make up 99% or more of B tree blocks. Excluding non-leaf blocks, the accessed metadata blocks reduce to: $m4$, $m5$, $m6$. Since each leaf block contains tuples, $m4$ appears multiple times, even though the upper layer doesn't access any data more than once. This is an example of correlated reference – accessing $m4$ multiple times within a short interval does not mean it is a hot block. This leads to an important observation: even if there are no correlated references in the data cache, its corresponding metadata cache may contain correlated references that must be properly handled by the cache replacement algorithm.

While we use B tree as an example to analyze access patterns of the metadata cache, the correlated reference access pattern is common across most metadata structures. A typical metadata block contains multiple smaller entities, such as key-value tuples, bits in a bitmap, or inodes in an inode block. This makes metadata blocks more susceptible to correlated references than data blocks: even if the upper layer visits different entities only once, the metadata block containing these entities is accessed multiple times, leading to correlated references. Thus, we argue that having correlated references is an inherent trait of metadata caches.

2.3 Deriving Metadata Traces

Trace-driven simulation is a crucial step in evaluating cache replacement algorithms. One challenge in improving replacement algorithms for metadata caches is the lack of available metadata traces. It is straightforward to collect data traces, as they capture the requests that go into storage systems. However, collecting metadata traces is much harder, since the storage system itself has to be instrumented to record each access to metadata blocks. Moreover, each access to a data block may trigger multiple accesses to internal metadata blocks, resulting in much longer metadata traces. Therefore, despite the recent release of multiple open-source cache datasets, all of them are limited to data traces.

To address the lack of metadata traces, we propose a method to derive metadata traces from any existing data traces. Using Figure 1 as an example, if each B tree leaf block stores 100 tuples mapping LBN to PBN, we can estimate that LBN 0-99 is in leaf block $m0$, LBN 100-199 are in leaf block $m1$, and so on. With this mapping, we can convert an LBN in the data trace to a metadata block by dividing it by the fan-out: $\frac{LBN}{FanOut}$. For example, a data cache trace

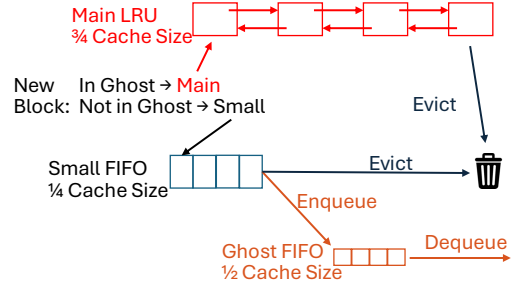


Figure 2: Illustration of 2Q cache replacement algorithm.

with requests for 1, 5, 107, and 720 would be converted into a metadata trace of 0, 0, 1, and 7. This approach allows us to generate metadata traces from existing data traces.

3 CLOCK2Q+ DESIGN

Given the similarities between the four cache replacement algorithms: 2Q [22], Clock2Q (the previous cache replacement algorithm in vSAN), S3-FIFO [53], and Clock2Q+, we first present and analyze the first three algorithms before introducing the design of Clock2Q+.

3.1 2Q

The 2Q [22] algorithm, illustrated in Figure 2, divides the cache into two parts: a Main LRU of $\frac{3}{4}$ of the cache size, which manages blocks using LRU, and a Small FIFO of $\frac{1}{4}$ of the cache size, which manages blocks using a FIFO queue. There is also a Ghost FIFO that stores a number of block entries equal to $\frac{1}{2}$ of the cache capacity. The Ghost FIFO only stores block numbers rather than actual data, so its memory usage is small compared to the cache size.

When a new block is accessed, if it is not found in the Ghost FIFO, it enters the Small FIFO. If the Small FIFO is full, its last block is evicted, and its block number is added to the Ghost FIFO, which may in turn evict the last block number from the Ghost FIFO if it is also full.

If a block number is found in the Ghost FIFO, it means that the block is actually hot but was mistakenly evicted from the Small FIFO previously. In this case, the block is added to the Main LRU. Following the LRU eviction policy, it may evict a block from the tail of the LRU. The Ghost FIFO provides the cache with a form of “long-term memory” of recently evicted blocks, which is particularly useful when workloads exhibit large loops in access patterns.

If an existing block in the Small FIFO is accessed, no action is needed. If the existing block is accessed in the Main LRU, it follows the standard LRU policy: it is removed from the LRU list and re-inserted at the head of the LRU list.

The Small FIFO serves the purpose of removing correlated references, as such references typically occur while the blocks are still in the Small FIFO, and they do not have a chance to be promoted to the Main LRU.

There are several drawbacks to 2Q:

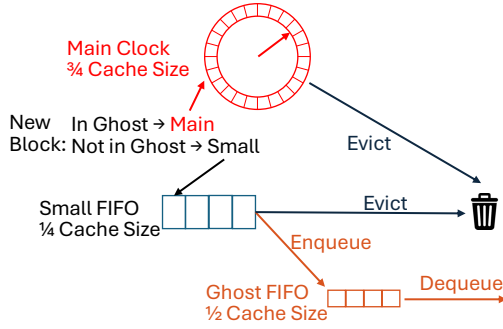


Figure 3: Illustration of Clock2Q replacement algorithm.

- *High LRU overhead.* The Main LRU has high CPU and memory overhead. So, it is hard to be made concurrent.
- *Small FIFO is too big.* The Small FIFO is designed to remove correlated references that typically occur within a short timeframe. However, 25% of the cache is allocated to the Small FIFO, a design choice made 30 years ago when caches were much smaller. In today’s storage systems, caches often exceed 10 gigabytes, and 25% of it could hold more than half a million blocks. Even under highly concurrent workloads, with thousands of threads loading cold blocks into the cache, it is unlikely that such a large Small FIFO is necessary to filter out correlated references. Reducing the Small FIFO size would allow for a larger Main LRU, potentially improving the overall miss ratio.
- *Unnecessary miss for hot blocks.* To enter the Main LRU, a block must first be evicted from Small FIFO and placed in the Ghost FIFO. This causes extra misses for blocks that are actually hot. In a large cache with low miss ratio, this introduces a non-trivial miss ratio increase.

3.2 Clock2Q

Clock2Q is the previous algorithm used in vSAN. As shown in Figure 3, the Clock2Q algorithm addresses the first limitation of 2Q by replacing the Main LRU with a Main Clock [44]. This modification lowers the CPU and memory overhead and was used in vSAN OSA, earlier versions of VDFS and vSAN ESA [44, 45]. Clock2Q still has the other two issues of 2Q (oversized Small FIFO and extra misses for hot blocks).

3.3 S3-FIFO

S3-FIFO is the state-of-the-art algorithm [53]. As shown in Figure 4, it is structurally similar to Clock2Q, but the Small FIFO now incorporates a Ref bit to track whether a block has been referenced while it is in the Small FIFO. The size of each cache component is also adjusted: the Main Clock size is increased from 75% to 90%, the Small FIFO size is reduced from 25% to 10%, and the Ghost FIFO size is increased from 50% to 100% of the cache size.

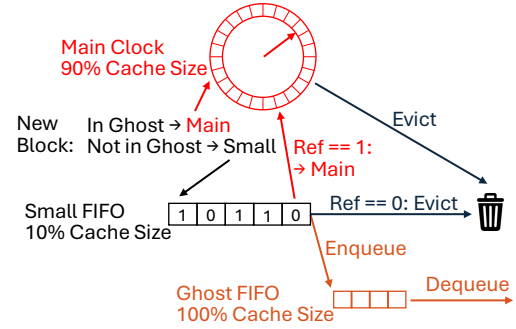


Figure 4: Illustration of S3-FIFO replacement algorithm.

S3-FIFO uses the Ref bit in the Small FIFO to remove the extra miss caused by hot blocks in 2Q and Clock2Q. When a block is accessed again after it enters the Small FIFO queue, its Ref bit is set. Later when this block is evicted from the Small FIFO queue, it will be moved into the Main Clock directly because its Ref bit is set. The block can bypass the Ghost FIFO and be directly promoted to the Main Clock, eliminating the unnecessary eviction step in 2Q and Clock2Q.

S3-FIFO still uses the Ghost FIFO to identify blocks that are evicted from the Small FIFO but later re-referenced.

S3-FIFO addresses all three issues of 2Q discussed in §3.1. However, it overcorrects the problem of the oversized Small FIFO by failing to handle correlated references altogether. This is caused by the Ref bit in the Small FIFO. If a cold block in the Small FIFO is accessed more than once due to correlated references, its Ref bit will be set. As a result, the block is wrongly promoted to the Main Clock and displace other hot blocks. This is not a major issue for data caches, where correlated references are rare. However, it may not work well on metadata cache, which has a lot more correlated references.

3.4 Clock2Q+

As shown in Figure 5, Clock2Q+ is based on Clock2Q and S3-FIFO, while addressing all the previously mentioned issues. The main difference between Clock2Q+ and S3-FIFO is the introduction of a “Correlation Window” in the Small FIFO. The Correlation Window is set to 50% of the Small FIFO size, or 5% of the total cache size. When a block in the Small FIFO is accessed, if it is still within the Correlation Window (i.e., within the Correlation Window size of blocks from the head of the Small FIFO), its Ref bit will not be set. This ensures that if correlated references occur within this short period, the block will not be misclassified as hot. After the block moves beyond of the Correlation Window, the subsequent access sets the Ref bit, making sure that it will be directly promoted to the Main Clock when it reaches the end of the Small FIFO.

Additionally, Clock2Q+ reduces the Ghost FIFO size to 50% of the total cache size, as empirical evaluations show that this performs as well as the previous 100% allocation while reducing memory overhead.

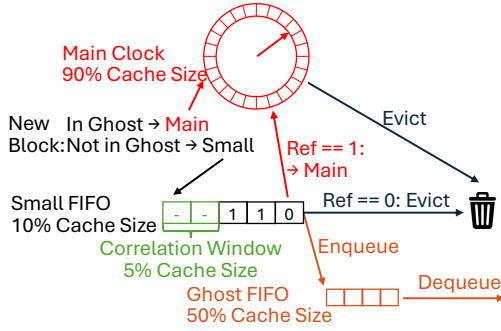


Figure 5: Illustration of Clock2Q+ replacement algorithm.

4 MAKING CLOCK2Q+ PRODUCTION READY

Before Clock2Q+, Clock2Q served as the core cache replacement algorithm used in vSAN and vSAN File Services for several years. The improved Clock2Q+ algorithm has been implemented and will be included in future releases. This section discusses the engineering efforts required to make the cache replacement algorithm production-ready.

4.1 Implementation

To reduce memory fragmentation and enhance performance, Clock2Q+ eliminates the need for memory allocation after cache initialization. Each of the Main Clock, Small FIFO, and Ghost FIFO is represented as an array within a contiguous block of memory. Arrays of cache entries are allocated for the Main Clock and Small FIFO, and an array of cache keys is allocated for the Ghost FIFO.

To support the hash table for cache entry lookup, an array of pointers to cache entries is allocated as the hash buckets for the cache data, in both the Main Clock and Small FIFO. Similarly, an array is allocated for the hash buckets for entries in the Ghost FIFO. Each cache entry contains a next-pointer for to handle hash collisions.

4.1.1 Concurrency. A cache is often accessed by multiple threads concurrently, making efficient thread safety an essential design requirement. Because of the simple data structures in Clock2Q+ (chained hash tables, FIFO queues, and arrays), ensuring concurrency is straightforward.

A lightweight mutex lock is added to each hash bucket for the chained hash table. The lock is held while operating on the linked list pointed by this bucket. Since the load factor is low (< 1), the average length of the linked list remains short. A per-bucket lock provides a fine-grained protection. Our in-house performance tests have never shown lock contention on these locks.

For cache entries in the Main Clock and Cold FIFO, a lightweight mutex lock is assigned to each entry. The lock is held briefly while the entry is read or updated, as the operations are fast in-memory computations. When an entry is evicted, and a disk I/O operation is issued to bring new content into this entry, the entry is marked as “Doing I/O”

and then inserted into the hash table. The per-entry lock is then released during the whole duration of the I/O. If another thread attempts to access the same block and finds it “Doing I/O”, it will wait on the entry until the I/O finishes.

Because each entry has a hash bucket lock and an entry lock, the lock ordering must be consistent to avoid deadlock. However, different operations require different lock orders:

- *Hash lock first* for cache lookup: A cache lookup must first lock the hash table to find the entry and then lock the cache entry.
- *Entry lock first* for new entry insertion: When a new entry is added, the entry is first locked to update its status. Then this entry is inserted into the hash table, at which point the hash bucket must be locked. Similarly, when an entry is evicted, the entry is locked first, followed by locking the hash bucket for removal from the hash table.

Clock2Q+ follows the “entry lock first” order. On every cache lookup, after the hash bucket lock is obtained and the entry is found, the hash bucket lock is released before the cache entry lock is taken. After the entry is locked, it is checked to ensure it has the expected key. Otherwise, it will unlock the entry and retry. This flow is shown in Figure 6. In line 9, after the entry is found in the hash table, this entry is evicted before it is locked in line 7. By comparing the key of the entry in line 8, we can identify the race and retry. If the entry cannot be found during the retry, it is treated as a cache miss.

```

1  CacheGetEntry(key)
2  {
3    retry:
4      LockHashBucket();
5      entry = SearchHash(key);
6      UnlockHashBucket();
7      LockEntry(entry);
8      if (entry.key != key) { // lost race
9        UnlockEntry(entry);
10       goto retry;
11     }
12     work on the entry;
13     UnlockEntry(entry);
14     ...
15  }
```

Figure 6: Lock Order Handling for Cache Lookup

Another part of the concurrency design involves the head and tail in the Small FIFO and the Ghost FIFO, and the clock hand in Main Clock. Since the Small and Ghost FIFO queues are always full, a single index is used to indicate both their head and tail. All these indices are atomic variables. Multiple threads can atomically increment these variables so that they traverse through the FIFO or Clock array to find eviction candidates or add new entries into the FIFO.

4.1.2 Debugging. The race condition presented in §4.1.1 is rare in real-world systems. However, it is important to test and verify its behavior. We developed a race enforcement

framework that can control any thread to pause on a “break-point” and resume when needed. We have a unit test using this framework that pauses one thread between line 6 and 7 in Figure 6, and let the second thread evict the cache entry the first thread is attempting to access. This forces the first thread to enter the race condition in lines 9 and 10, enabling us to test the handling of the race condition.

4.1.3 Dirty Pages. The cache must handle dirty pages, which are often managed by Write-Ahead Logs. Before log entries are flushed to persistent storage, the corresponding dirty pages described by these log entries cannot be written out [18]. Even if the dirty pages can be written out and thus chosen as candidates for eviction, flushing them requires an I/O operation, making them much slower to evict compared to clean pages. Therefore, Clock2Q+, as used in vSAN and vSAN File Services, skips dirty pages when selecting eviction candidates. This is equivalent to reinserting the dirty page at the head of the Main Clock or the Small FIFO.

Another issue related to dirty pages arises when moving hot pages from the Small FIFO to the Main Clock. If a dirty page at the end of the Small FIFO has its Ref bit set, it should be moved to the Main Clock. Although it is technically possible to move the page, the current implementation leaves it in the Small FIFO. Since the implementation uses an array instead of linked lists for both the Small FIFO and the Main Clock, moving a page between them requires copying rather than simple pointer updates. Furthermore, the transaction system tracks dirty pages using a dirty list for future flushing. Copying dirty pages is challenging because it requires locking the dirty list during the copy process. The dirty page remains in the Small FIFO until it is revisited in a later cycle and is no longer dirty. The impact of this change on the overall miss ratio is expected to be small. §5.5.1 evaluates and confirms that this change has a negligible effect on cache performance.

Since the Small FIFO is small, but the number of dirty pages can be large, there is a risk that, for some workloads, every page in the Small FIFO could be dirty. The default algorithm could enter an infinite loop, searching for an evictable page in the entire Small FIFO and failing. To address this, instead of scanning the entire Small FIFO, the algorithm scans only a fixed number of entries. If all scanned entries are dirty, it skips the remaining entries and checks the Main Clock for eviction candidates.

In our system, two flushing mechanisms are implemented to handle dirty pages:

- (1) Time-based flushing: A time threshold is set. If dirty pages remain in the cache longer than this threshold, they are flushed.
- (2) Proportion-based flushing: The cache maintains low and high watermark thresholds for the proportion of dirty pages. If the proportion of dirty pages in the cache exceeds the high watermark, dirty pages are flushed in order, from oldest to newest, until the proportion drops below the low watermark.

4.2 Live Cache Resizing

A generic storage system like vSAN must support a wide range of workloads. Some workloads have large working set sizes and require a larger cache than the default size. Dynamic cache growth and shrinking are therefore built into the system so that the cache can handle requests without disruption while it is being grown or shrunk.

The cache resizing mechanism must meet the following criteria:

- Live: All regular cache operations must continue during resizing.
- Low CPU overhead: The additional CPU usage caused by resizing should not affect cache performance.
- Minimal data movement: The majority of the cache data should remain in place during resizing.
- Fine granularity: Cache size adjustments should be incremental, growing and shrinking by small amounts each time, avoiding large-scale reallocations.
- No memory fragmentation: Frequent cache resizing should not lead to fragmented system memory.

4.2.1 Cache Growth. When the cache grows, all data structures in the cache must also grow, including arrays for the Small FIFO, Main Clock, and Ghost FIFO. During cache initialization, sufficient unused virtual address space is reserved at the end of each array. This allows the arrays to grow without requiring relocation. The first step in cache growth is to allocate additional space at the end of all these arrays.

After the newly grown entries are initialized, the cache size and hash bucket size are updated to reflect the new cache size. A background thread then begins traversing all cache entries, moving them from the old location to the new location. Each cache entry may reside in either the old or the new location during this process. Searching through two hash locations correctly is challenging because cache entries are not locked during lookups, and the background thread can move them at any point during the search. Our design shifts this complexity from lookup time to insertion time.

When a cache lookup occurs, it searches only the new hash location. If a false negative occurs because the entry still resides in the old hash location, the insertion routine will detect it, rehash it, and move it to the new location. Subsequently, the retry logic described in §4.1.1 will redo the lookup and succeed. This design simplifies the handling of race conditions: while cache entries are not locked during lookups, both cache entries and hash buckets are locked during insertion. As a result, locking both hash locations, searching, and moving entries becomes more manageable.

4.2.2 Cache Shrinking. Cache shrinking follows a similar approach to cache growth. The background thread first rehashes all cache entries into the new, smaller hash buckets and then frees the extra memory at the end of the hash bucket arrays. Data in the Ghost FIFO queue is easier to shrink because entries at the end of the array can be discarded.

Data at the end of the Main Clock and Small FIFO arrays must be handled differently. Clean pages can be discarded; however, dirty pages cannot be discarded. As discussed in §4.1.3, copying dirty pages in the cache is not straightforward. When a dirty page is encountered, the background thread triggers a transaction flush and retries later, with the hope that the page will be flushed and clean by the next check.

4.3 Lessons Learned

A cache replacement algorithm is a critical component of the data path in a storage system. Many storage products and modules at VMware require effective cache replacement algorithms. The Clock2Q+ algorithm originated from a simple Clock2Q algorithm and evolved over the years, shaped by the needs of various products and requirements. It ultimately developed into Clock2Q+ and is now consolidated into a single source codebase, shared across multiple products.

In 2015, a new feature in vSAN OSA required a CPU-efficient, scan-resistant cache replacement algorithm for the core data path running in the OS kernel. The expected cache hit ratio was close to 100%, making the low CPU overhead of a Clock-based algorithm a suitable choice. The code operates as a state machine on a single core, so there was no need to handle locking or concurrency. Since the algorithm runs in kernel mode, debugging is challenging. Therefore, selecting a simple algorithm was a high priority. Additionally, the algorithm needed to be scan-resistant, as certain workloads periodically scan all blocks.

Although multiple scan-resistant Clock-based algorithms, such as Clock-Pro [20] and CAR/CART [2], are available, none of them was simple enough for consideration. The 2Q algorithm is one of the simplest algorithms that is also scan-resistant. However, it relies on LRU, which has a higher CPU overhead.

At this point, the Clock2Q design became straightforward: replace the LRU in 2Q with Clock, as Clock has been shown to behave similarly to LRU. Almost 10 years later, the S3-FIFO paper demonstrated that Clock performs better than LRU in many cases, which came as a pleasant surprise: Clock2Q may have a lower miss ratio and lower CPU and memory overhead than 2Q!

Another benefit of Clock, compared to Clock-Pro or CAR/CART, is that it does not require adjusting the size of the Clock or inserting/deleting elements in the middle of the Clock. As a result, arrays, instead of linked lists, can be used for the Main Clock, Small FIFO, and Ghost FIFO, which reduces memory overhead and further simplifies the implementation.

Around the same time, another new project, VDFS, also required a cache algorithm. Since multiple instances of the cache were needed to store different types of data, a C++ template was used to generalize the cache implementation. Clock2Q was chosen again for its simplicity.

In this use case, concurrency was required because the cache was accessed by multiple threads. We started with a simple solution of using one giant lock to protect the entire

cache. Although the lock was released when reading data from the disk during a cache miss, the giant lock still caused high lock contention. The next solution we tried was sharding. Sharding resolved the lock contention until the project's transaction system required reserving cache blocks so that each transaction had enough free cache space to finish without deadlocking the system. However, because blocks needed by a transaction could go to any shard, if one transaction needed N blocks, it was only safe to reserve N blocks in each shard, which was too wasteful to be practical.

We designed the highly concurrent version of Clock2Q discussed in §4.1.1 to replace sharding. After this change, cache access was no longer a performance bottleneck.

Since vSAN supports a wide variety of workloads, its cache must support online resizing with minimal data movement. Because Clock2Q uses only arrays and a chained hash table, this simplicity enables us to support growth and shrinking with minimal changes.

After S3-FIFO was published, its similarity to Clock2Q became the most important factor that attracted our team to look deeper. While studying S3-FIFO, we began the work described in this paper with an initial hunch that adding a correlation window in the Small FIFO could combine the best aspects of Clock2Q and S3-FIFO and outperform both.

As the cache supports various workloads, a bug was discovered: when every page in the Small FIFO is dirty, the algorithm enters an infinite loop. This inspired us to study the impact of limiting the maximum number of cache entries the Cold FIFO head or clock hand needs to visit before it gives up, which is discussed in §5.5.2.

When S3-FIFO was implemented in vSAN, we found that dirty blocks needed to be moved from the Small FIFO to the Main Clock, which was never necessary in Clock2Q. We chose simplicity over accuracy, as discussed in §4.1.3.

The above evolution of Clock2Q+ illustrates a typical process of how an academic algorithm is adapted to real systems. Minimal engineering effort is applied at each stage to avoid premature optimization. An algorithm with balanced characteristics (simplicity, reasonable hit ratio, low CPU overhead, high concurrency, etc.) is desired. Among all these traits, simplicity is of paramount importance in production systems, although it is rarely discussed in academia. A cache replacement algorithm is a type of algorithm where simplicity is especially critical. If there is a bug in the cache replacement algorithm, it is hard to determine whether it is a bug or the algorithm simply does not work well under that workload. It is even harder to reproduce and debug in production systems. This is because real workloads are not easily accessible, and it is difficult to set up and run repeatable workloads with realistic access patterns. Commonly available benchmarks use simple distributions (such as uniform random or Zipf), which do not represent realistic access patterns. In our case, the initial choice of a simple design proved to be highly beneficial, especially when cache resizing was implemented and tested many years later.

5 EVALUATION

In this section, we discuss Clock2Q+ in relation to all five goals listed in §1 and focus on evaluating its miss ratio against other algorithms.

Clock2Q+ has the same low CPU overhead as any clock-based algorithm on a cache hit, requiring only a hash lookup and a Ref bit update. It also has high multi-core scalability because every cache entry and hash bucket has a separate lock. In any performance testing in vSAN ESA and VDFS, cache lock contention never appeared as a bottleneck. The implementation of Clock2Q+ is straightforward because it only uses a few arrays (for the Clock, FIFOs, and hash buckets) and singly linked lists (for hash table collisions). This makes it simpler than any other scan-resistant algorithm. Additionally, Clock2Q+ requires no manual tuning, as its default parameter is robust (see §5.6).

In this section, we compare Clock2Q+ to state-of-the-art cache replacement algorithms on different workloads, including metadata, data, and non-block workloads. We study the impact of dirty blocks and evaluate whether the simplifications in §4.1.3 are effective. We further analyze why it achieves higher cache efficiency. Additionally, we study the sensitivity of its parameters to ensure its robustness.

5.1 Experiment Setup

Workloads. Since vSAN is deployed on enterprise customers' hardware, we cannot conduct experiments using customers' data due to privacy and compliance reasons. Therefore, we used open-source block cache datasets from CloudPhysics [42]. The CloudPhysics dataset contains 106 traces spanning 7 days, with 2,114 million requests for 82 TB of data. Since Clock2Q+ is designed for metadata caching in vSAN, and none of the open-source traces were collected from metadata caches, we derived metadata traces from the production data traces using the method described in §2.3: dividing the LBNs in the data traces by 200. The fan-out of 200 was chosen to match the fan-out of the real B tree in vSAN ESA.

Benchmarks. We implemented Clock2Q+ in the cache simulator libCacheSim [52]. This ensures a fair and accurate comparison with state-of-the-art algorithms. We evaluated different cache replacement algorithms at four different cache sizes: 0.005, 0.01, 0.05, and 0.1 of the trace footprint.

Baseline and Metrics. We compared Clock2Q+ with state-of-the-art replacement algorithms. Since previous work has shown that Clock achieves both lower overhead and a lower miss ratio than LRU [51], we used Clock as the baseline and calculated the miss ratio improvement as

$$\text{Improvement}_{\text{algo}} = \left(\frac{\text{MR}_{\text{Clock}} - \text{MR}_{\text{algo}}}{\text{MR}_{\text{Clock}}} \right) \quad (1)$$

5.2 Metadata Trace Fidelity

We validate the realism of our metadata trace produced by our simple derivation method by generating and comparing two metadata traces. In the first trace, we created a B tree using the TLX library [4], replayed a data trace, and recorded the accessed leaf blocks. In the second trace, we applied

the method described above: for each request in the data trace, we divided the LBN by 200. We evaluated S3-FIFO and Clock2Q+ on the two traces. Figure 7 shows that all differences are less than 0.01% between the two traces. This indicates that the proposed method of dividing LBN by fan-out is an effective approach for deriving metadata traces from data traces.

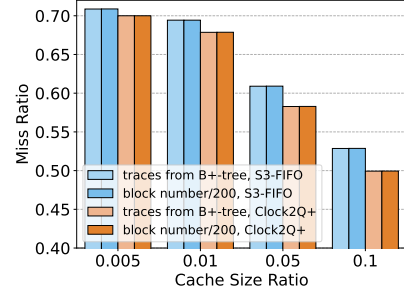


Figure 7: Clock2Q+ and S3-FIFO show almost identical miss ratios on derived metadata traces and traces collected from TLX B tree leaf block accesses.

5.3 Miss Ratio Improvements

Metadata traces. Figure 8a compares Clock2Q+ with ten state-of-the-art cache replacement algorithms on the metadata traces. We observe that most algorithms do not significantly outperform Clock; that is, they do not show positive improvements in the figure. This is because these algorithms struggle to handle correlated references effectively. As a result, blocks with correlated references are often misclassified as hot, leading to suboptimal eviction decisions in these algorithms.

We find that only S3-FIFO 2-bit (default S3-FIFO), 2Q, and Clock2Q+ demonstrate significant improvements across the two cache sizes. S3-FIFO 2-bit uses a 2-bit frequency counter. Blocks in S3-FIFO 2-bit need to be re-referenced at least twice before they can be moved to the Main Clock, which effectively reduces the likelihood of cold blocks with correlated references entering the Main Clock. In comparison, S3-FIFO 1-bit admits blocks with only a single re-reference from the Small FIFO to the Main Clock. As a result, it is more susceptible to correlated references and exhibits a higher miss ratio compared to S3-FIFO 2-bit. In 2Q, blocks in the Small FIFO are always evicted without being reinserted into the Main Clock. This approach also addresses the correlated reference problem, as the frequency of block access does not influence this behavior.

Clock2Q+, as described in §3.4, is specifically designed to handle workloads with correlated references, utilizing the correlation window filter to prevent cold blocks from being moved into the Main Clock. Among all algorithms, Clock2Q+ achieves the best miss ratio improvement across all tested cache sizes. For the larger cache size, S3-FIFO 2-bit performs second-best, and Clock2Q+ outperforms it by up to 28.5% in miss ratio under this setup.

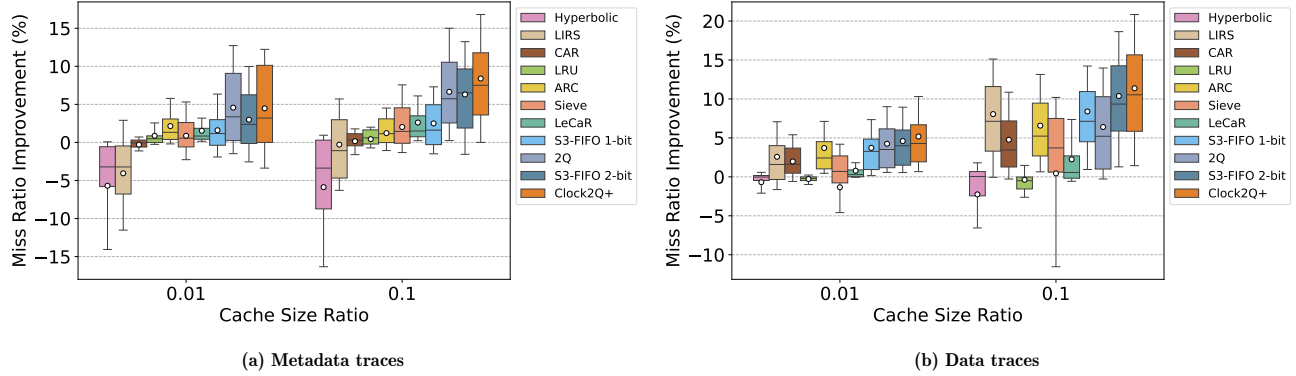


Figure 8: Comparing Clock2Q+ with state-of-the-art eviction algorithms.

Data traces. For comparison, we evaluated the same set of algorithms on data traces. Figure 8b shows that most of the state-of-the-art algorithms provide substantial miss ratio improvements compared to the baseline (Clock). This differs from the previous observation in Figure 8a but aligns with expectations, as these algorithms are designed for data cache workloads. While the level of improvement varies across different replacement algorithms, Clock2Q+ still demonstrates the highest improvement among all. Despite being designed for metadata cache workloads, Clock2Q+ also performs exceptionally well on data cache workloads. Furthermore, this suggests that some data cache workloads may also exhibit correlated references, highlighting the broader applicability of Clock2Q+.

Consistent improvement across cache sizes. Figure 9 summarizes the miss-ratio curves for the CloudPhysics w56 workload. The left shows metadata and the right row shows data. Within each subplot, the four curves compare Clock, ARC, S3-FIFO, and Clock2Q+ on the same workload and object set.

Across all four panels, Clock2Q+ consistently has the best curve: it lies strictly below the existing algorithms over on the **entire** cache-size range (from 1% to 100%). On metadata traces, Clock2Q+ provides a visible downward shift in the curve. For data blocks, the absolute miss ratios are higher. Here Clock2Q+ again has the lowest miss ratios over entire range of cache sizes.

In summary, Clock2Q+ consistently achieves lower miss ratios than state-of-the-art algorithms, with the improvement being more pronounced in the metadata traces.

5.4 The Secret Behind High Efficiency

To understand the high efficiency of Clock2Q+, we explore the differences in blocks inserted into the Main Clock and the Ghost FIFO in S3-FIFO and Clock2Q+. We use one trace as a case study throughout this section. We plotted the Probability Density Function (PDF) of the next reuse distance (NRD) of blocks moved to the Main Clock and to the Ghost FIFO from the Small FIFO. A small NRD indicates that the block will be accessed soon after leaving the Small

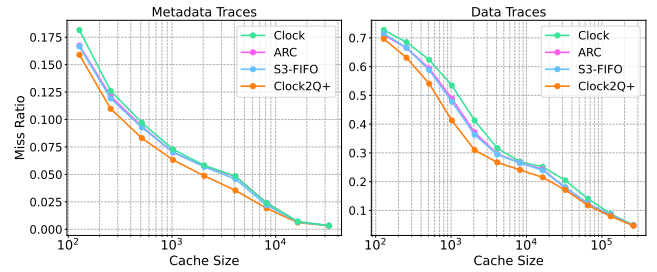


Figure 9: Miss-ratio curves for CloudPhysics w56. The x-axis is cache size (log scale, unit is byte), and the y-axis is miss ratio. Lower curves indicate better performance (fewer misses) at a given cache size.

FIFO and should ideally stay in the Main Clock to reduce cache misses. A large NRD suggests that the block will not be reused soon and can be evicted.

Figure 10a shows that, compared to S3-FIFO, Clock2Q+ moves a larger portion of blocks with *small* NRD to the Main Clock, ensuring these hot blocks remain in the cache longer. Figure 10b shows that fewer such blocks end up in the Ghost FIFO, reducing the likelihood of mistakenly evicting blocks that will be reused soon. The results demonstrate that Clock2Q+ makes better decisions by keeping hot blocks in the Main Clock and evicting cold blocks to the Ghost FIFO.

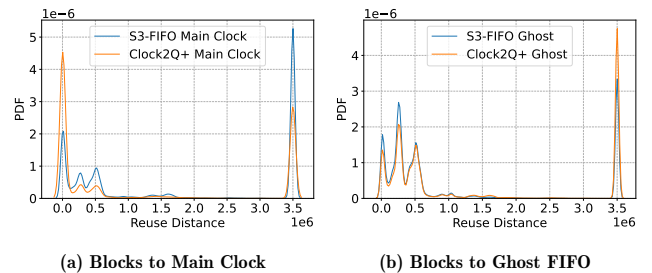


Figure 10: PDF of Next Reuse Distance (NRD) of blocks from Small FIFO. Clock2Q+ moves more cold blocks (large NRDs) to Ghost FIFO and moves more hot blocks (small NRDs) to the Main Clock. The rightmost peak represents blocks that are not accessed again.

Table 1 shows the number of blocks that Clock2Q+ and S3-FIFO moved into the Main Clock and the Ghost FIFO from the Small FIFO. Compared to S3-FIFO, Clock2Q+ moves significantly fewer blocks (less than $\frac{1}{4}$) from the Small FIFO into the Main Clock. At the same time, the number of blocks moved from the Cold FIFO to the Ghost FIFO is about the same between the two algorithms. These findings highlight the importance of selecting the correct blocks to move from the Small FIFO to the Main Clock, as this decision will impact the miss ratio. By being more selective about what enters the Main Clock, Clock2Q+ reduces “pollution” in the Main Clock. This suggests that the correlation window of Clock2Q+ is an effective mechanism for deciding which blocks should be promoted to the Main Clock when evicting from the Small FIFO.

Table 1: Counts of inserted block in the Main Clock and Ghost FIFO for Clock2Q+ and S3-FIFO.

Algorithm	Small → Main	Small → Ghost	Ghost → Main
Clock2Q+	20,762	186,959	33,068
S3-FIFO	88,529	126,897	29,909

5.5 Impact of Production Optimizations

5.5.1 Dirty Blocks. In Clock2Q+, when the Small FIFO searches for an eviction candidate, it skips dirty blocks. After a threshold of dirty blocks is skipped, it gives up searching in Small FIFO and directly inserts the new block into the Main Clock. This avoids a dead loop when all blocks in the Small FIFO are dirty.

As discussed in §4.1.3, when a dirty block is evicted from the Small FIFO, if its Ref bit is set, it should be moved to the Main Clock. However, implementing such movement in vSAN is not trivial. Therefore, the dirty block is re-inserted into the Small FIFO and will be checked again when it becomes the eviction candidate next time. Whether and by how much such optimizations for handling dirty blocks impact the miss ratio is investigated in this section.

We set the dirty block flush time to 30 seconds. The low and high watermarks for flushing dirty blocks are set to 10% and 20% of cache size, respectively, which are the same as the values¹ used in the Linux kernel page cache.

Figure 11 shows the impact of simplified dirty block handling on 106 CloudPhysics traces. A negative value indicates that the miss ratio is increased for the trace. We observe that the simplified dirty block handling has minimal impact on most traces. At the smallest cache size, we observe that skipping dirty blocks can reduce the miss ratio for some traces. We conjecture that when the cache size is very small, the Small FIFO is not large enough, causing some potentially hot blocks to be evicted from the Small FIFO. Therefore, skipping dirty blocks in the Small FIFO helps avoid evicting

those hot blocks. At larger cache sizes, we observe that simplified dirty block handling slightly increases the miss ratio for some traces. However, overall, the impact is negligible. This observation justifies the simplification of dirty block handling in real systems.

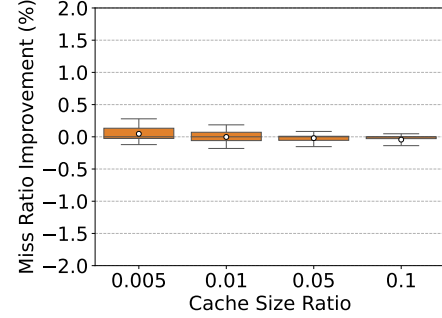
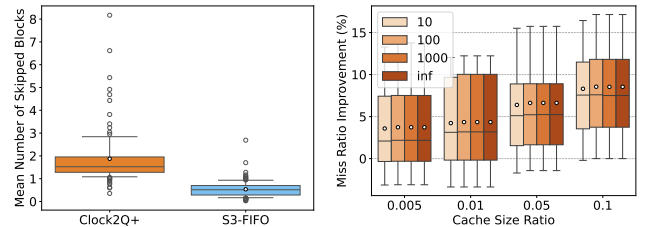


Figure 11: Impact of simplified dirty block handling. Miss ratio improvement is calculated against Clock2Q+ that moves dirty pages from the Small FIFO to the Main Clock.

5.5.2 Limiting Clock Hand Movement. When evicting from the Main Clock, a block can be skipped if its Ref bit is set. The number of skipped blocks can be high during a single eviction, which can potentially lead to increased CPU usage. Figure 12a shows the mean number of skipped blocks across the 106 CloudPhysics traces. We observe that it is low (<2) for most traces for both Clock2Q+ and S3-FIFO. Compared to S3-FIFO, Clock2Q+ has more skipped blocks per eviction. This is likely because Clock2Q+ keeps more hot blocks in the Main Clock, which results in more skipped blocks.

To limit the number of skipped blocks, we set a threshold per eviction at 10, 100, 1000, and ∞ (no restriction). Once the reinsertion limit is reached, a block is forcibly evicted from the Main Clock if it is not dirty, regardless of its Ref bit value. Figure 12b shows the differences between the various thresholds. We observe that limiting the maximum number of skipped blocks to 10 only slightly increases the miss ratio. This suggests that limiting the number of skipped blocks to a small value, e.g., 10, is a safe choice for production systems.



(a) Mean skipped block count distribution (b) Impact of limiting the number of skipped blocks per eviction

Figure 12: Impact of reinsertion limit on miss ratio.

¹dirty_background_ratio and dirty_ratio in /proc/sys/vm/, respectively

5.6 Sensitivity and Versatility Analysis

Clock2Q+ uses half of the Small FIFO as the correlation window size. In this section, we evaluate the impact of the correlation window size. Figure 13 shows the results for three different correlation window sizes (10%, 30%, and 50% of the Small FIFO). At small cache sizes, a larger correlation window slightly improves the miss ratio; however, the improvement is limited at larger cache sizes. Our conjecture is that the correlation window size is an inherent property of the workload and should not vary significantly with cache sizes. Therefore, at small cache sizes, a larger correlation window is beneficial. Overall, the results indicate that the miss ratio is not very sensitive to the correlation window size, as all configurations provide significant miss ratio improvements over the baseline.

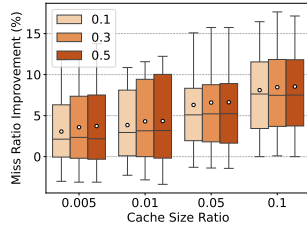


Figure 13: Impact of correlation window size.

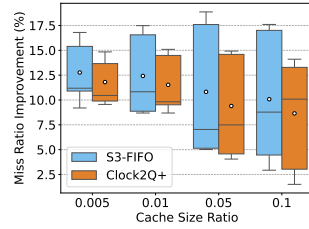


Figure 14: Miss ratio improvements on non-block traces.

So far, we have only evaluated block traces and have shown that Clock2Q+ achieves a lower miss ratio than state-of-the-art algorithms on both derived metadata traces and the original data traces. We also explore the efficiency of Clock2Q+ on non-block cache workloads. We used three open-source cache datasets [53], namely, the Wikimedia, Meta Key-Value, and Tencent Photo datasets. In total, there are 10 traces. Figure 14 shows that the miss ratio improvement of Clock2Q+ is slightly smaller than that of S3-FIFO on this dataset of key-value and object traces. We conjecture that this is because these workloads have limited correlated references, and the correlation window in Clock2Q+ does not provide enough benefit. Instead, it causes some potentially hot objects to be evicted from the Small FIFO.

6 RELATED WORK

6.1 Cache Replacement Algorithms

Cache replacement algorithm is a research area that spans decades with numerous literature [7, 15, 21, 22, 28, 40, 41, 50, 54]. LRU is the most well-known and widely used algorithm because it is easy to understand and implement [14]. Clock approximates LRU with a lower overhead and a lower miss ratio [13, 53]. Both LRU and Clock are not scan-resistant.

Many advanced algorithms were designed to make the cache scan-resistant. LIRS [21] uses the distance between the last two references of a block to determine whether a block is hot. LIRS2 [55] improves this by using the sum of the distance to a block’s last two recent accesses. Clock-Pro [20] is a Clock variation of LIRS where the CPU overhead

of cache hit is small by setting a Ref bit during cache hit. DLIRS [27] dynamically tunes LIRS’ parameters to make it work well in different workloads. ARC [28] uses two queues with adaptive size to store blocks with recency and frequency. CAR and CART are the Clock variations of ARC [2]. LRU-K [29] uses the last k th access to a block to determine a block’s hotness. When K is 1, it becomes LRU. SEQ [17] detects long sequential accesses and handles them differently. SLRU [19, 24] split the LRU list into two parts: one holds blocks accessed only once, and one holds hot blocks. EELRU [23] keeps track of block numbers of blocks that are just evicted out of the cache to detect sequential scan or large loop patterns and evict blocks earlier to improve the hit ratio. LFU [37] uses the frequency to determine the hot blocks that should stay in the cache. It tends to keep once frequently accessed blocks in the cache, even after they become cold later. Variations of LFU such as LFU* [1], LFU-Aging [31], and LFU*-Aging [47] are proposed to overcome this problem. FBR [31] is similar to LFU but does not count correlated references. LRFU [26] combines recency and frequency into one parameter and presents a spectrum of algorithms in between LRU and LFU. LeCaR [41] uses machine learning to determine the weight between LRU and LFU based on workload. However, LeCaR is not scan-resistant. Cacheus [34] solves this by learning between two variations of LRU and LFU (SR-LRU and CR-LFU). GDSF [8] considers data size together with frequency for eviction. TinyLFU uses a Bloom Filter and the Count Min Sketch to count the number of references to blocks before admitting them into the main cache [15]. The *Mr. cache* algorithm can be applied to any algorithm by splitting a cache into multiple regions to reduce the CPU contention [43]. LRB [38], HALP [39], and 3L-cache [56] use machine learning to rank objects at eviction. GL-Cache [50] uses machine learning to rank object groups for group-level evictions. LHD [3] uses probability to decide eviction candidates. Hyperbolic [5] uses frequency over age as a metric to rank eviction candidates. S3-FIFO [53] uses a small FIFO queue to quickly filter out unpopular objects before they enter the main Clock queue. SIEVE [54] is a new Clock-like algorithm that uses a different hand movement to achieve “quick demotion” and “lazy promotion”. Some algorithms use application hints to make replacement decisions, such as DEAR [9], AFC [10], UBM [25], ILRU and OLRU [35], Hot Set [36], and QLSM [11].

6.2 Correlated References

Correlated references were identified in the work of FBR [31]. Later, multiple algorithms were designed to identify and filter out correlated references. 2Q [22] uses the Small FIFO to filter them out. However, it evicts all the blocks exiting the FIFO queue, causing one cache miss for every hot block in the main Clock. CART [2] is a variation of CAR but can handle correlated references. Its idea is similar to 2Q because it uses the length of T_1 (similar to the small FIFO of 2Q) as the correlation window size. If a block is found on one of its ghost queue B_1 , the block is marked as outside of the

correlation window. This is similar to 2Q where an extra miss is generated for hot blocks. Clock2Q+ uses a better approach than CART and 2Q by not generating extra misses when identifying blocks outside of the correlation window.

7 CONCLUSION

Metadata cache workloads often exhibit correlated references. We present a simple and effective cache replacement algorithm, Clock2Q+, designed for metadata caches and implemented in multiple production systems at VMware. Clock2Q+ achieves a lower miss ratio on metadata traces than state-of-the-art cache replacement algorithms and provides up to a 28.5% lower miss ratio than S3-FIFO. Besides a lower miss ratio, Clock2Q+ also has low CPU and memory overhead, is highly concurrent, requires no tuning, and is easy to implement, which is one of the most important characteristics desired in a production system.

REFERENCES

- [1] Martin F. Arlitt. 1996. *A performance study of Internet web servers*. Master's thesis. Department of Computer Science, University of Saskatchewan.
- [2] Sorav Bansal and Dharmendra S. Modha. 2004. CAR: Clock with Adaptive Replacement. In *Proceedings of the 3rd USENIX Conference on File and Storage Technologies* (San Francisco, CA) (FAST '04). USENIX Association, USA, 187–200.
- [3] Nathan Beckmann, Haoxian Chen, and Asaf Cidon. 2018. LHD: Improving cache hit rate by maximizing hit density. In *15th USENIX symposium on networked systems design and implementation (NSDI'18)*. 389–403.
- [4] Timo Bingmann. 2018. TLX: Collection of Sophisticated C++ Data Structures, Algorithms, and Miscellaneous Helpers. <https://panthema.net/tlx/>, retrieved Oct. 7, 2020.
- [5] Aaron Blankstein, Siddhartha Sen, and Michael J. Freedman. 2017. Hyperbolic caching: Flexible caching for web applications. In *2017 USENIX annual technical conference (ATC'17)*. USENIX Association, Santa Clara, CA, 499–511.
- [6] Daniel P. Bovet and Marco Cesati. 2005. *Understanding the Linux Kernel* (3 ed.). O'Reilly Media, Inc.
- [7] Richard W. Carr and John L. Hennessy. 1981. WSCLOCK: a simple and effective algorithm for virtual memory management. In *Proceedings of the eighth ACM symposium on Operating systems principles (SOSP'81)*. Association for Computing Machinery, New York, NY, USA, 87–95. <https://doi.org/10.1145/800216.806596>
- [8] Ludmila Cherkasova. 1998. *Improving WWW proxies performance with greedy-dual-size-frequency caching policy*. Citeseer.
- [9] Jongmoo Choi, Sam H. Noh, Sang Lyul Min, and Yookun Cho. 1998. An adaptive block management scheme using on-line detection of block reference patterns. In *Proceedings of the 1998 International Workshop on Multimedia Database Management Systems (IW-MMDBMS 1998)*. Dayton, OH, 172–179.
- [10] Jongmoo Choi, Sam H. Noh, Sang Lyul Min, and Yookun Cho. 2000. Towards application/file-level characterization of block references: A case for fine-grained buffer management. In *Proceedings of the 2000 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS 2000)*. Santa Clara, CA, 286–295.
- [11] Hong-Tai Chou and David J. DeWitt. 1985. An evaluation of buffer management strategies for relational database systems. In *Proceedings of the 11th International Conference on Very Large Data Bases (VLDB 1985)*. Stockholm, Sweden, 174–188.
- [12] Alex Conway, Abhishek Gupta, Vijay Chidambaram, Martin Farach-Colton, Rick Spillane, Amy Tai, and Rob Johnson. 2020. SplinterDB: closing the bandwidth gap for NVMe key-value stores. In *Proceedings of the 2020 USENIX Conference on Unix Annual Technical Conference (USENIX ATC'20)*. USENIX Association, USA, Article 4, 15 pages.
- [13] Fernando J. Corbat and Victor A. Vyssotsky. 1969. A Paging Experiment with the Multics System. *Honeywell Inc.* (1969).
- [14] Wolfgang Effelsberg and Theo Hrder. 1984. Principles of Database Buffer Management. In *ACM Transactions on Database Systems (TODS)*, Vol. 9. ACM, 560–595. Issue 4.
- [15] Gil Einziger, Roy Friedman, and Ben Manes. 2017. TinyLFU: A Highly Efficient Cache Admission Policy. *ACM Transactions on Storage* 13, 4 (Dec. 2017), 1–31. <https://doi.org/10.1145/3149371>
- [16] Kevin W. Froese and Richard B. Bunt. 1996. The effect of client caching on file server workloads. In *Proceedings of the 29th Hawaii International Conference on System Sciences (HICSS 1996)*. ACM, Kihei, HI, USA, 150–159.
- [17] Gideon Glass and Pei Cao. 1997. Adaptive page replacement based on memory reference behavior. In *Proceedings of the 1997 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS 1997)*. ACM, Seattle, WA, USA, 115–126.
- [18] Theo Haerder and Andreas Reuter. 1983. Principles of transaction-oriented database recovery. *ACM Computing Surveys (CSUR)* 15, 4 (1983), 287–317.
- [19] Qi Huang, Ken Birman, Robbert van Renesse, Wyatt Lloyd, Sanjeev Kumar, and Harry C. Li. 2013. An analysis of Facebook photo caching. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles (SOSP'13)*. Association for Computing Machinery, New York, NY, USA, 167–181. <https://doi.org/10.1145/2517349.2522722>
- [20] Song Jiang, Feng Chen, and Xiaodong Zhang. 2005. Clock-Pro: An Effective Improvement of the Clock Replacement. *Proceedings of the USENIX Annual Technical Conference* (2005), 323–336.
- [21] Song Jiang and Xiaodong Zhang. 2002. LIRS: an efficient low interference recency set replacement policy to improve buffer cache performance. In *ACM SIGMETRICS Performance Evaluation Review (SIGMETRICS'02, Vol. 30)*. 31–42. <https://doi.org/10.1145/511399.511340>
- [22] Theodore Johnson and Dennis Shasha. 1994. 2Q: A Low Overhead High Performance Buffer Management Replacement Algorithm. In *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB'94)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 439–450.
- [23] Yannis Smaragdakis Scott Kaplan and Paul Wilson. 1999. EELRU: Simple and effective adaptive page replacement. In *Proceedings of the 1999 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS 1999)*. ACM, Atlanta, GA, USA, 122–133.
- [24] Ramakrishna Karedla, Spencer Love, and B. Wherry. 1994. Caching strategies to improve disk system performance. *Computer* 27, 3 (1994), 38–46. <https://doi.org/10.1109/2.268884>
- [25] Jong Min Kim, Jongmoo Choi, Jesung Kim, Sam H. Noh, Sang Lyul Min, Yookun Cho, and Chong Sang Kim. 2000. Unified buffer management scheme that exploits sequential and looping references. In *Proceedings of the 4th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2000)*. San Diego, CA.
- [26] Donghee Lee, Jongmoo Choi, Jong-Hun Kim, Sam H. Noh, Sang Lyul Min, Yookun Cho, and Chong Sang Kim. 2001. LRFU: A Spectrum of Policies that Subsumes the Least Recently Used and Least Frequently Used Policies. *IEEE Trans. Comput.* 50, 12 (2001), 1352–1361.
- [27] Cong Li. 2018. DLIRS: Improving Low Inter-Reference Recency Set Cache Replacement Policy with Dynamics. In *Proceedings of the 11th ACM International Systems and Storage Conference, SYSTOR 2018, HAIFA, Israel, June 04-07, 2018*. ACM, 59–64. <https://doi.org/10.1145/3211890.3211891>
- [28] Nimrod Megiddo and Dharmendra S Modha. 2003. ARC: A self-tuning, low overhead replacement cache. In *2nd USENIX conference on file and storage technologies (FAST'03)*.
- [29] Elizabeth J. O'Neil, Patrick E. O'Neil, and Gerhard Weikum. 1993. The LRU-K Page Replacement Algorithm For Database Disk Buffering. In *Proceedings of the 1993 ACM SIGMOD international conference on Management of data*. ACM, 297–306.
- [30] Hal Pomeranz. [n.d.]. Understanding EXT4 (Part 3): Extent Trees. <https://web.archive.org/web/20190818050155/https://digital-forensics.sans.org/blog/2011/03/28/digital-forensics-understanding-ext4-part-3-extent-trees>.
- [31] John T. Robinson and Murthy V. Devarakonda. 1990. Data cache management using frequency-based replacement. In *Proceedings of the 1990 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS 1990)*. ACM, Boulder, CO, USA, 134–142.

- [32] Ohad Rodeh. [n.d.]. COW B-trees. <https://github.com/ohad-rodeh/bt>.
- [33] Ohad Rodeh, Josef Bacik, and Chris Mason. 2013. BTRFS: The Linux B-Tree Filesystem. *ACM Transactions on Storage* 9, 3 (2013), 1–32.
- [34] Liana V. Rodriguez, Farzana Yusuf, Steven Lyons, Eysler Paz, Raju Rangaswami, Jason Liu, Ming Zhao, and Giri Narasimhan. 2021. Learning Cache Replacement with CACHEUS. In *19th USENIX Conference on File and Storage Technologies (FAST’21)*. USENIX Association, 341–354.
- [35] Giovanni Maria Sacco. 1987. Index access with a finite buffer. In *Proceedings of the 13th International Conference on Very Large Data Bases (VLDB 1987)*. Brighton, England, 301–309.
- [36] Giovanni Maria Sacco and Mario Schkolnick. 1986. Buffer management in relational database systems. *ACM Transactions on Database Systems (TODS)* (1986).
- [37] Daniel D. Sleator and Robert E. Tarjan. 1985. Amortized Efficiency of List Update and Paging Rules. *Commun. ACM* 28, 2 (1985), 202–208.
- [38] Zhenyu Song, Daniel S Berger, Kai Li, Anees Shaikh, Wyatt Lloyd, Soudeh Ghorbani, Changhoon Kim, Aditya Akella, Arvind Krishnamurthy, Emmett Witchel, and others. 2020. Learning relaxed belady for content distribution network caching. In *17th USENIX symposium on networked systems design and implementation (NSDI’20)*. 529–544.
- [39] Zhenyu Song, Kevin Chen, Nikhil Sarda, Deniz Altinbuken, Eugene Brevdo, Jimmy Coleman, Xiao Ju, Pawel Jurczyk, Richard Schooler, and Ramki Gummadi. 2023. HALP: Heuristic Aided Learned Preference Eviction Policy for YouTube Content Delivery Network. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI’23)*. 1149–1163.
- [40] Linpeng Tang, Qi Huang, Wyatt Lloyd, Sanjeev Kumar, and Kai Li. 2015. RIPQ: Advanced photo caching on flash for facebook. In *13th USENIX Conference on File and Storage Technologies (FAST’15)*. 373–386.
- [41] Giuseppe Vietri, Liana V. Rodriguez, Wendy A. Martinez, Steven Lyons, Jason Liu, Raju Rangaswami, Ming Zhao, and Giri Narasimhan. 2018. Driving cache replacement with ML-based LeCaR. In *10th USENIX workshop on hot topics in storage and file systems (hotStorage’18)*. USENIX Association, Boston, MA.
- [42] Carl A. Waldspurger, Nohhyun Park, Alexander Garthwaite, and Irfan Ahmad. 2015. Efficient MRC construction with SHARDS. In *13th USENIX conference on file and storage technologies (FAST’15)*. USENIX Association, Santa Clara, CA, 95–110.
- [43] Wenguang Wang. 2004. *Storage Management for Large Scale Systems*. Ph. D. Dissertation. Department of Computer Science, University of Saskatchewan.
- [44] Wenguang Wang. 2017. System and methods of a CPU-efficient cache replacement algorithm. U.S. Patent 9760493B1. Filed on March 14, 2016, issued on September 12, 2017.
- [45] Wenguang Wang, Mounesh Badiger, Abhay Kumar Jain, Junlong Gao, Zhaohui Guo, and Richard P. Spillane. 2021. System and method of a highly concurrent cache replacement algorithm. U.S. Patent 11086779B2. Filed on November 11, 2019, issued on August 10, 2021.
- [46] Wikipedia. [n.d.]. XFS. <https://en.wikipedia.org/wiki/XFS>.
- [47] Darryl L. Willick, Derek L. Eager, and Richard B. Bunt. 1993. Disk cache replacement policies for network filesystems. In *Proceedings of the 13th International Conference on Distributed Computing Systems (ICDCS 1993)*. Pittsburgh, PA, 2–11.
- [48] Gala Yadgar, Michael Factor, Kai Li, and Assaf Schuster. 2011. Management of Multilevel, Multiclient Cache Hierarchies with Application Hints. *ACM Transactions on Computer Systems* 29, 2 (May 2011), 5:1–5:51. <https://doi.org/10.1145/1963559.1963561>
- [49] Gala Yadgar, Michael Factor, and Assaf Schuster. 2007. Karma: Know-it-All Replacement for a Multilevel Cache. In *5th USENIX Conference on File and Storage Technologies (FAST’07)*. (FAST’07).
- [50] Juncheng Yang, Ziming Mao, Yao Yue, and K. V. Rashmi. 2023. GL-Cache: Group-level learning for efficient and high-performance caching. In *21st USENIX Conference on File and Storage Technologies (FAST’23)*. (FAST’23). 115–134.
- [51] Juncheng Yang, Ziyue Qiu, Yazhuo Zhang, Yao Yue, and K. V. Rashmi. 2023. FIFO can be Better than LRU: the Power of Lazy Promotion and Quick Demotion. In *Proceedings of the 19th Workshop on Hot Topics in Operating Systems (HOTOS’23)*. Association for Computing Machinery, New York, NY, USA, 70–79. <https://doi.org/10.1145/3593856.3595887>
- [52] Juncheng Yang, Yao Yue, and K. V. Rashmi. 2020. A large scale analysis of hundreds of in-memory cache clusters at Twitter. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI’20)*. USENIX Association, 191–208. <https://www.usenix.org/conference/osdi20/presentation/yang>
- [53] Juncheng Yang, Yazhuo Zhang, Ziyue Qiu, Yao Yue, and Rashmi Vinayak. 2023. FIFO queues are all you need for cache eviction. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP’23)*. Association for Computing Machinery, New York, NY, USA, 130–149. <https://doi.org/10.1145/3600006.3613147>
- [54] Yazhuo Zhang, Juncheng Yang, Yao Yue, Ymir Vigfusson, and K. V. Rashmi. 2024. SIEVE is Simpler than LRU: an Efficient Turn-Key Eviction Algorithm for Web Caches. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI’24)*. 1229–1246.
- [55] Chen Zhong, Xingsheng Zhao, and Song Jiang. 2021. LIRS2: an improved LIRS replacement algorithm. In *Proceedings of the 14th ACM International Conference on Systems and Storage (Haifa, Israel) (SYSTOR’21)*. Association for Computing Machinery, New York, NY, USA, Article 19, 12 pages. <https://doi.org/10.1145/3456727.3463772>
- [56] Wenbin Zhou, Zhixiong Niu, Yongqiang Xiong, Juan Fang, and Qian Wang. 2025. 3L-Cache: Low Overhead and Precise Learning-based Eviction Policy for Caches. In *23rd USENIX Conference on File and Storage Technologies (FAST’25)*. USENIX Association, Santa Clara, CA, 237–254. <https://www.usenix.org/conference/fast25/presentation/zhou-wenbin>